

The Complete Walkthrough

One document that explains everything, end to end: how a Forward Deployed Engineer works (the eight steps), how our system is built (the architecture), and how the application we made answers the four questions the panel will judge it by. Written so that anyone — technical or not — can follow it.

Project: Meridian Pipeline Intelligence · Data as of May 2, 2026 · Prepared by Sai Saran

The three parts

PART ONE — The FDE way of working

1. What is a Forward Deployed Engineer?
2. Step 1 — Requirement gathering
3. Step 2 — Choosing the model and AI tools
4. Step 3 — Implementation
5. Step 4 — RAG (and why we said no to it)
6. Step 5 — Front-end integration
7. Step 6 — Deployment
8. Step 7 — Support and observability
9. Step 8 — Documentation

PART TWO — The architecture

10. The big idea: the restaurant
11. The five lanes, one by one
12. The ten guardrails in simple words
13. The five data problems, in simple words

PART THREE — The application, judged by the four questions

14. Question 1 — Did we answer a real business question, under time pressure?
 15. Question 2 — Did we make defensible cuts?
 16. Question 3 — Trust and traceability: walking a skeptical CCO through it
 17. Question 4 — What does it prove, and what does it not?
 18. Likely questions, short answers
 19. The one-minute closing speech
- Appendix A — The numbers you must know
- Appendix B — Simple glossary

1 · What is a Forward Deployed Engineer?

A Forward Deployed Engineer — FDE for short — is an engineer who works **at the client's side**, not back at headquarters. A normal engineer builds a product and many customers use it. An FDE goes to one customer, understands their exact problem, and builds something that solves it — quickly, and in a way the customer can trust and use.

Because every client is different, FDEs follow a repeatable set of steps so nothing important is missed. The eight steps are:

1. **Requirement gathering** — understand what is needed, what the tool should and should not do.
2. **Choosing the model and AI tools** — pick the right technology for the job.
3. **Implementation** — build it.
4. **RAG** — if the assistant needs to read documents, connect it to them. (We will explain this — and why we chose not to use it.)
5. **Front-end integration** — give people a screen to talk to it.
6. **Deployment** — package it so it runs anywhere, reliably.
7. **Support and observability** — watch it while it runs; keep records of what it does.
8. **Documentation** — write everything down before moving on.

The next eight chapters take each step in turn. For each one you get: what the step means in simple words, exactly what we did in this project, and a sentence you can say in the meeting.

SAY IT LIKE THIS

I followed the standard FDE delivery lifecycle end to end — requirements, tool selection, implementation, integration, deployment, observability, and documentation — and I can show you evidence for each step. The one step I deliberately skipped is RAG, and I'll explain exactly why skipping it was the right engineering call for this data.

2 · Step 1 — Requirement gathering

IN SIMPLE WORDS

Before you build anything, find out what is actually needed. Who will use the tool? What questions must it answer? What must it never do? Writing this down first prevents building the wrong thing fast.

In a normal engagement you would sit with the product team and ask questions. In this project, the "client" spoke to us through a written brief — so we squeezed every drop of meaning out of it.

WHAT WE DID IN THIS PROJECT

The scope — Meridian Systems, a company that sells software, has sales data spread across places that don't talk to each other. Leaders wait days for simple answers. We were asked to build a tool where a sales leader types a question in plain English and gets an answer.

The objectives — every answer must have three parts: a direct answer (no dodging), the actual data rows behind it (proof, not promises), and a clear warning whenever the answer relies on incomplete data or an assumption.

The guardrails — this was the special part. Meridian tried an AI tool before. It invented numbers, and the Chief Commercial Officer (CCO — the top sales boss) caught it doing so in a meeting. So the hidden requirement, more important than any feature, was: **this tool must be impossible to catch lying**. We turned that into ten written guardrails (Chapter 12).

The concrete targets — three questions the tool must handle: how is the Enterprise segment tracking against quota; which salespeople are at risk of missing this quarter; and how does this quarter compare to last quarter at the same point in time.

One extra thing we did that wasn't asked: before building, we studied the data itself, line by line — and found five serious problems in it (Chapter 13). Requirement gathering isn't only listening to people; it's also listening to the data.

SAY IT LIKE THIS

The written requirement was "answer three questions." The real requirement, reading the story, was "win back a CCO who was burned by AI." Every design decision I made traces back to that second, unwritten requirement.

3 · Step 2 — Choosing the model and AI tools

IN SIMPLE WORDS

Pick the technology that fits the problem — the programming language, the libraries, and which AI model to use. Good engineers also decide where **not** to use AI.

WHAT WE DID IN THIS PROJECT

Language and libraries: Python (the standard language for data work), **pandas** (a library that works with tables of data, like a very powerful spreadsheet), **Streamlit** (turns Python into a web page quickly — perfect for a prototype), and Plotly for charts.

The AI model: we built the app so the AI part is a plug. It ran first on Anthropic's Claude, then on Google's Gemini — switched in one file, with zero change to any safety property. That was deliberate: it proves our safety lives in **our code**, not in any vendor's model. No lock-in.

The most important selection decision was not which model — it was where the model is allowed to work. We gave the AI exactly two jobs: (1) understand which of our eight approved questions the user is asking, and (2) turn a finished, computed answer into friendly sentences. It is never allowed to compute a number. This decision came straight from the requirement: a model that can't touch numbers can't invent them.

SAY IT LIKE THIS

The headline tool decision wasn't Claude versus Gemini — we ran both, and swapping took one file. The real decision was where AI is allowed at all: it understands questions and phrases answers; it never computes. That's what makes the last vendor's failure impossible here.

4 · Step 3 — Implementation

IN SIMPLE WORDS

Now build it. Good implementation keeps different jobs in different files, so each part can be checked — and replaced — on its own.

WHAT WE DID IN THIS PROJECT

The application is about two thousand lines of Python in six clean pieces:

- `config.py` — every date, threshold and assumption on one page. An auditor reads this single file and knows all the rules.
- `data_loader.py` — reads the client's files, checks them, and refuses to continue if anything is wrong. Also automatically discovers the five data problems.
- `analytics.py` — the calculator. Eight analyses, all plain arithmetic. **No AI can reach this file.** Every answer it produces carries its own evidence, formula, warnings, and a trust grade.
- `question_router.py` — matches a question to one of the eight analyses using simple, readable rules.
- `llm_layer.py` — the only file that talks to AI, including the validator that checks every AI sentence.
- `app.py` — the screen. It only displays; it computes nothing.

An honest difference from the standard playbook: the classic FDE lifecycle builds a separate API with FastAPI. We didn't — for a prototype, that's an extra moving part with no extra proof. But the calculator is already shaped like an API (every answer is one tidy package), so extracting it behind FastAPI is week one of the production plan, not a rewrite.

Alongside the app we wrote **42 automated tests** — small programs that check our program. Every important number was first calculated by hand from the raw files, then locked into a test. If anything changes, a test fails loudly.

SAY IT LIKE THIS

The build rule was: every number must be born in one file that AI cannot reach, and every answer must leave that file carrying its own evidence. Everything else — the chat, the AI, the charts — is wrapping around that core.

5 · Step 4 — RAG, and why we said no to it

IN SIMPLE WORDS

RAG stands for Retrieval-Augmented Generation. In plain words: before the AI answers, the system **fetches relevant documents** (from a special search database, like ChromaDB) and hands them to the AI to read. It's the right tool when answers live inside **documents** — manuals, policies, contracts, support articles.

The standard lifecycle includes RAG because most assistants answer from documents. So why did we skip it? Because of what our data **is**.

WHAT WE DID IN THIS PROJECT

Meridian's data is not documents. It is **structured tables** — rows and columns of deals, dollar values, and dates. Questions like "what is Enterprise attainment?" are not answered by *finding a similar paragraph*; they are answered by *filtering rows and adding numbers*.

RAG works by similarity search — "find me text that looks related to this question." Similarity is fuzzy by nature. Fuzzy retrieval feeding an AI is exactly how you get an answer that is *plausible but wrong* — the precise failure that burned this client. For tables, retrieval should be **exact**: a filter, not a guess.

So our "grounding" — the thing that ties answers to real data — is stronger than RAG for this job: deterministic queries, the actual source rows attached to every answer, and a validator that blocks any number the calculator didn't produce.

When we would add RAG: the day Meridian brings unstructured text — contracts, call notes, sales playbooks. Then RAG becomes analysis number nine: built, tested, and audited like the other eight, with citations back to the exact document passage.

SAY IT LIKE THIS

We skipped RAG on purpose, not by omission. RAG grounds answers in documents; our data is tables, where grounding should be an exact query, not a similarity search. Bolting a vector database onto numeric data is how you make confident wrong answers — the one thing this client cannot forgive.

6 · Step 5 — Front-end integration

IN SIMPLE WORDS

Give users a screen. In the standard playbook this is a chat widget embedded in an existing website. The goal is the same everywhere: meet users where they are, and make the tool feel effortless.

WHAT WE DID IN THIS PROJECT

There is no existing Meridian website in this scenario, so the front-end **is** the application — a clean, light, chat-style web page. What's on it, top to bottom:

- **Four big cards** that appear the moment the page loads, counting up with a small animation: the quarter's target (\$6.2M), what's actually been won (\$518K), what's still being worked (\$5.2M), and the gap that remains even in the best case (\$484K). The state of the quarter, before anyone asks anything.
- **Five suggested question buttons** — one click each for the questions leaders ask most.
- **A chat box** for anything else, in ordinary English.
- **Every answer as a card**: a colored trust grade, the answer in one or two sentences, the key numbers as tiles, a chart, any warnings, and three tabs — the source rows, the exact calculation, and the assumptions.
- **A sidebar** where the tool is honest about itself: data checks passed, the AI on/off switch, the five known data problems, a live activity log, and the full methodology.

Design choices were deliberate too: light, calm, no clutter — a tool for an executive, not a toy. In production, the same answer "package" can be rendered inside any existing portal; the screen is the replaceable part.

SAY IT LIKE THIS

The front-end has one design goal: nothing on this screen asks to be trusted — everything on it shows why it can be. Even the layout is an argument: the receipts are one click away on every single answer.

7 • Step 6 — Deployment

IN SIMPLE WORDS

Packaging the app so it runs the same way on any machine. **Docker** puts the app and everything it needs into a sealed box called a container. **Docker Compose** starts that box with one command. "It works on my laptop" becomes "it works anywhere."

WHAT WE DID IN THIS PROJECT

We containerized the application. One command — `docker compose up` — builds and starts it, with the AI keys passed in as settings and the activity log stored outside the box so it survives restarts.

Two touches worth pointing out, because they carry the trust idea into deployment:

- **The build refuses to produce a broken image.** All 42 tests run *inside* the build. If the data fails validation or any number is wrong, there is no image at all. You cannot ship a bad build even by accident.
- **The container checks its own health** every 30 seconds, so an operations team knows immediately if it stops responding.

SAY IT LIKE THIS

Deployment follows the same rule as everything else: fail closed. The Docker build runs the entire test suite — if the data doesn't reconcile, the image simply cannot be built. A broken version can't reach users, because it can't even reach existence.

8 · Step 7 — Support and observability

IN SIMPLE WORDS

Once a tool is live, someone must be able to see what it's doing — how many questions, how fast, any failures — and to look back at what it said if a question ever comes up. That record-keeping is called observability.

WHAT WE DID IN THIS PROJECT

The audit log. Every question asked of the tool is written to a permanent log file: the time, the question, how it was understood, what trust grade the answer carried, whether the AI's wording passed validation, and how long it took. If anyone ever asks "what did the tool tell people last Tuesday?" — the answer exists.

Live stats on screen. The sidebar shows the running counts: questions logged, answered versus refused, AI summaries verified, average response time — plus the most recent questions. Observability isn't hidden in a server room; it's part of the product's honesty.

Self-monitoring. The five data checks run at every start; the container health-check runs every 30 seconds; and the logging itself is built to never crash the app — if the log can't be written, the tool keeps answering.

Support posture. When the AI provider had a real outage during our build (a billing failure), the app degraded gracefully — it kept answering with its own wording, and users saw no error. We watched our own "support story" work live.

SAY IT LIKE THIS

Every answer this tool has ever given is in an audit log, with its trust grade and its validation result. In production that log feeds the monitoring stack — but even on a laptop, accountability is already built in.

9 · Step 8 — Documentation

IN SIMPLE WORDS

Before moving to the next assignment, write everything down — so the client's own team can run, trust, and extend the work without you. Documentation is what makes a hand-off possible.

WHAT WE DID IN THIS PROJECT

The documentation kit, all in the project:

- **README** — what it is, how to run it (locally or with Docker), and an index of everything else.
- **One-page summary** (leave-behind PDF) — architecture, the three answers, the five findings. For executives.
- **Presenter's guide** (24 pages) — a full plain-English walkthrough of the product, paced for an hour.
- **Guardrails specification** (4 pages) — the ten safety constraints: what each prevents, where it lives in the code, how to demonstrate it.
- **Architecture diagram** (1 page) — the whole system in one picture, with the guardrails pinned where they act.
- **This walkthrough** — the end-to-end story.
- **Inside the app itself** — a methodology panel with every metric definition and assumption. The documentation lives where the users are.
- **The tests as documentation** — 42 tests that state, in runnable form, exactly what every number should be. Documentation that checks itself.

SAY IT LIKE THIS

My rule for documentation: the client's own team should be able to run it, trust it, and extend it without me in the room. If I'm still needed to explain it, the documentation isn't finished.

10 · The big idea: the restaurant

Every architecture conversation about this system can start with one picture: a restaurant.

The AI is the **waiter**. It listens to what you ask for, works out which item from the menu you mean, and later describes the finished dish in a pleasant sentence. But the waiter **never cooks**.

The **kitchen** is ordinary, checkable computer code — the same arithmetic a careful analyst would do in a spreadsheet, but automated, tested, and double-checked. Only the kitchen produces numbers.

The **menu is short on purpose**: eight dishes — eight kinds of questions — each one prepared, tasted, and audited before it was ever served to a customer. If you ask for something not on the menu, the waiter says so honestly and shows you the menu, instead of improvising a dish that might poison someone.

And there is one more character the last restaurant didn't have: a **food inspector** standing between the kitchen and your table. Before any AI-phrased answer reaches you, the inspector checks every number in the sentence against what the kitchen actually produced. One number that doesn't match — the whole sentence is thrown out and replaced with the kitchen's own plain wording, and you're told this happened.

SAY IT LIKE THIS

The failure of the last tool was that one system was both waiter and cook — so every answer was a potential fabrication. We separated the jobs permanently. The model chooses from a menu and phrases results. It never cooks. And an inspector checks every plate.

11 • The five lanes, one by one

The architecture diagram (docs/architecture.pdf) shows five horizontal lanes. A question falls through them top to bottom. Color tells you who works where: **green is deterministic code** — the "kitchen," where AI cannot reach; **blue is the AI** — exactly two boxes, both followed by checks; **red is the refusal path** — drawn as a first-class destination, not an error.

Lane 1 — Interface

The sales leader and the chat screen. The screen only displays. It owns no logic and computes nothing — so there is nothing on it to distrust.

Lane 2 — Routing: choose from the menu, or refuse

First, simple keyword rules try to match the question to one of the eight analyses — rules a human can read in a minute. Questions about things the data can't support (next quarter, forecasts) are refused before anything else happens. If the rules can't place a question, the AI gets its first job: pick which menu item fits. Its choice is then re-checked against the menu, and any person or segment it names is re-checked against the real roster. Anything invalid is thrown away. If nothing fits — the red refusal box: "I can't answer that reliably," plus the list of what it can answer.

Lane 3 — The deterministic engine (the green lane)

The heart. The client's two data files are read and checked — wrong shape, unreadable value, or totals that don't reconcile five different ways, and the app refuses to serve anything at all. The same loading step automatically discovers the five data problems (F1–F5). Then the chosen analysis runs: plain filtering and arithmetic. It produces not just a number but a complete package — the answer, the source rows, the written calculation, the assumptions used, the warnings that apply, and a self-assigned trust grade.

Lane 4 — Narration: the model may phrase, never compute

The AI's second job: turn that package into two to four friendly sentences. Then the inspector — the numeric validator — extracts every number from those sentences and demands each one exist, exactly, in the package. One mismatch and the entire narration is rejected (visibly). Even sneaky changes are caught: writing "\$3.65M" instead of \$3,650,000 counts as a mismatch, because silently changing units is still distortion.

Lane 5 — Operations

The audit log records every question and answer. Docker packages everything, with the test suite as a gate on the build itself. The verification harnesses — 42 tests and a routing accuracy benchmark — live here too.

SAY IT LIKE THIS

If you remember one thing from the diagram, remember the colors: numbers are born only in the green lane, the model only ever touches the two blue boxes, and both blue boxes are followed by a check written in ordinary code.

12 · The ten guardrails in simple words

A guardrail is a safety constraint that keeps the assistant inside its intended job. Ours share one principle: **enforced by code, not by prompt**. We never ask the model to behave. We make misbehaving either impossible (there's no code path for it) or immediately visible.

	GUARDRAIL	IN SIMPLE WORDS
G1	Closed menu	The tool can only do its eight approved analyses. The AI picks from that list; its pick is re-checked. There is no way to talk it into a new calculation.
G2	Honest refusal	Questions about things the data can't support get "I can't answer that reliably" — plus the list of what it can do. Never a guess.
G3	Real names only	If the AI says the question is about a person or segment, that name must exist in the real roster, or the routing is discarded.
G4	AI can't touch numbers	Every figure comes from plain, tested code the model cannot reach. Not a rule it follows — a wall it can't cross.
G5	Fail closed	If the data fails any check, the app shows nothing rather than possibly-wrong numbers. Even the Docker build refuses to produce an image if tests fail.
G6	The number inspector	Every number in an AI sentence must match the computed results exactly, or the whole sentence is thrown out — and you're shown that it was.
G7	Self-grading answers	Every answer wears a color grade set by fixed rules — green, amber, or red ("a data problem dominates this; fix the data, don't act on the number").
G8	Receipts always	An answer cannot exist without its source rows, formula, and warnings attached. It's the only output format the system has.
G9	Never depends on the AI	If the AI is down, slow, or wrong, the app quietly answers in its own plain wording. The AI is a nicety, never a dependency — and swapping vendors changed nothing.
G10	Everything on record	Every question and answer is logged with its trust grade and validation result. Nothing the tool says is unaccountable.

SAY IT LIKE THIS

If a panelist asks "what if the model ignores its instructions?" — the answer is: it can ignore every instruction we gave it and still cannot put a wrong number on this screen. That's the difference between a guardrail written in a prompt and a guardrail written in code.

13 · The five data problems, in simple words

While loading Meridian's files, the tool compared the two snapshots — the closed Q1 books and the current Q2 system — record by record. Five kinds of problems appeared. They matter twice over: they change what the sales numbers mean, and finding them is the best proof that this tool checks instead of assuming. Together they touch **28 of the 92 deals**.

F1 — The same ticket number given to two different customers (13 deals · \$1.97 million)

Every deal has an ID number, like a ticket. Thirteen ticket numbers point to **completely different companies** in the two snapshots — in Q1, ticket OPP-076 was a small won deal for "Cascade Financial"; in Q2 the same ticket is a brand-new, unfinished deal for "Evergreen Capital." Someone deleted old records and reused their numbers. Why it matters: any "how did this deal change over time" comparison using these tickets is nonsense — and 38% of the entire open pipeline sits on them. What we do: treat the current records as real, ban them from history comparisons, and flag every answer they touch.

F2 — Finished deals that came back to life (7 deals · \$576 thousand)

Seven deals that were finished in the closed Q1 books — won or lost — are open again in the Q2 system, or flipped outcome. "Final" is supposed to mean final. Worse, some of their money is now counted twice in spirit: already in Q1's results, and also sitting in Q2's pipeline as if it could be won again. What we do: Q1 reporting follows the closed books; these deals count in Q2 only as open pipeline; the overlap is flagged.

F3 — Deals past their promised date (2 deals · \$87 thousand)

Two open deals are past the dates their salespeople predicted. Small housekeeping issue — kept, and flagged.

F4 — The same revenue counted in two quarters (3 deals · \$373 thousand — the big one)

Three deals are marked **won in both snapshots**, but their dates were moved: won in late March per the closed Q1 books, won in early April per the Q2 system. The same money appears in both quarters' results. Why it's the most serious finding: of the \$518,000 the system calls "won this quarter," \$373,000 — seventy-two percent — is this recycled money. Truly new Q2 sales: \$145,000. For Enterprise, the effect is total: its only "win" this quarter is one of these deals, so truly-new Enterprise revenue is **zero**. This also touches finance and pay: moving revenue across a closed quarter is the kind of thing auditors ask about. What we do: show both figures everywhere, and any answer dominated by this problem grades itself red.

F5 — History quietly edited (3 deals · \$260 thousand)

Three finished deals had their value or date changed after the books closed — one won deal went from \$155,000 to \$120,000 with no change in outcome and no explanation. Why it matters: anyone re-running "Q1 results" from the current system now gets different totals than the closed books — a guaranteed argument in a meeting unless someone surfaces it first. What we do: Q1 follows the Q1 file; the edits are shown, not silently adopted.

SAY IT LIKE THIS

We didn't find these problems by luck — the tool finds them automatically, every time it loads, and if Meridian fixes the source records, every flag clears itself. The most valuable thing on this screen might not be any sales number. It's the list of reasons to be careful about the sales numbers.

14 · Question 1 — Did we answer a real business question, under time pressure?

The brief asked for three questions at minimum. The tool answers all three — plus five more analyses — and each answer changed what a leader would actually do. That last part is what makes a question "real."

"How is Enterprise tracking against quota?"

Answer: \$165,000 won of a \$3,650,000 target — 4.5%, with 35% of the quarter gone. But the tool grades its own answer **red**, because that entire \$165,000 is a deal the closed Q1 books already counted (its date was moved). Truly new Enterprise revenue this quarter: **zero**. A dashboard would have said "4.5%" and moved on. This answer changes the Enterprise conversation entirely.

"Which reps are at risk of missing Q2?"

Answer: three people — Kevin Marsh, Lisa Park, Ryan Cole — are mathematically short: even winning every open deal wouldn't reach their target. Three others the tool **refuses to grade**, because most of their pipeline sits on corrupted records. That refusal protects careers — it's the difference between "bad quarter" and "bad data."

"How does Q2 compare to Q1 at the same point?"

Answer: fairly compared at day 32 of each quarter, Q2 has won \$518,000 versus Q1's \$1,260,000 — about 41% of last quarter's pace. And counting only truly-new deals, just \$145,000. Balanced with honesty: Q1 also started slow and finished at 102%, so the message is "create pipeline now," not "panic."

And one number nobody asked for

The most important figure in the dataset appears before any question is asked: **\$484,000 of the quarter's target is uncovered even if Meridian wins every single open deal** — and the earliest sales stage is completely empty, meaning nothing new is being seeded. That's a finding a CCO acts on the same day.

The time constraint, honestly

The estimated scope was one to two hours; the deterministic core — data checks, the three answers, evidence, tests — fits that scope. I then deliberately extended it (the AI layer with validation, deployment, observability, documentation) to show the production direction. I'll happily talk about which parts belong to which phase — the layering was the plan, not an accident.

SAY IT LIKE THIS

Every one of the three answers came back with a twist a dashboard would have missed — zero new Enterprise revenue, three unjudgeable reps, a pace gap that survives every fairness check. Answering the question is table stakes. Changing the decision is the job.

15 · Question 2 — Did we make defensible cuts?

Under time pressure, what you *don't* build matters as much as what you do. Every cut below follows one test: **does this trade verifiability for capability?** If a feature would make the tool more capable but harder to verify, it was cut — because for this client, unverifiable capability has negative value.

WHAT WE CUT	WHY — AND WHAT WE DID INSTEAD
Free-form "ask anything"	Letting the AI write its own database queries means infinite possible calculations — impossible to pre-audit, and the exact source of "plausible but wrong." Instead: eight audited analyses, growing one at a time.
RAG / vector database	Right tool for documents; wrong tool for tables. Retrieval on numeric data should be exact, not fuzzy. Added the day the client brings documents. (Chapter 5.)
Forecasting / "weighted pipeline"	Predicting which open deals will close requires historical stage-by-stage outcomes the data doesn't contain. Any probabilities would be invented numbers — the exact sin that burned this client. Instead: coverage measured against the <i>real</i> Q1 win rate (77.9%).
A separate API (FastAPI)	An extra moving part with no extra proof at prototype stage. The engine is already API-shaped; extraction is week one of production.
Login / user accounts	Security theater on sample data. Designed for production (single sign-on, role-based views), not faked in a demo.
Editing the client's data	The tool found five data problems; it flags them and never "fixes" them silently. Correcting the books is the client's decision, with finance in the room.
Corporate branding	Kept the design neutral — in a real engagement this tool would wear the client's brand, because it's their leaders' tool.

SAY IT LIKE THIS

My test for every cut was the same: does it trade verifiability for capability? If yes, it went — because for a client burned by a lying tool, capability you can't verify is worth less than nothing. And each cut has a written path back in, one audited step at a time.

16 · Question 3 — Trust and traceability: walking a skeptical CCO through it

This is the heart of the assignment. Not "is the code safe" — but "how would you stand in front of the person who was embarrassed by AI, and earn the next ten minutes of their attention?" Here is that walk, step by step. It works because at no point do we ask for belief; every step hands over control.

Minute one — open with the tool's own bad news

Don't start with a success. Start with the red badge: "I asked it your own question — how is Enterprise tracking — and I want you to see that its answer is *red*. The tool is telling us its own headline number can't be trusted, because it discovered your Q1 books and Q2 system share the same revenue." A tool that volunteers its own bad news is the opposite of the one that lied.

Minute two — hand over the receipts

Click the Source data tab. "That 4.5% comes from exactly one deal. Here's the row. Here's the formula — sum of won Enterprise deals divided by the four Enterprise reps' targets. You could recheck it on a napkin." Invite them to actually do that, once, with any number they choose. One verified napkin beats an hour of assurances.

Minute three — let them try to break it

"Ask it anything. Genuinely — anything." Off-menu questions hit an honest refusal. Future questions hit "no data exists." A misheard question shows its interpretation in plain sight at the top of the answer — the worst case is visible and correctable. The skeptic's instinct is to hunt for the lie; give them the hunt, because the architecture means there's no invisible place for a lie to live.

Minute four — show the inspector working

Point at the small line under an answer: "AI summary — 11 numbers verified against computed results." Then explain the veto: if the AI's sentence contained even one number the calculator didn't produce — even just "\$3.65M" as shorthand — the whole sentence would be discarded, and the screen would say so. "The model physically cannot put a number in front of you that the arithmetic didn't produce."

Minute five — the honest close

"I won't tell you this tool can't be wrong. My own first version passed every test and still missed the shared-revenue problem — I found it by attacking my own results, and now that check runs automatically. What I'll promise is different: when this tool is wrong, it's wrong *visibly, traceably, and fixably*. And everything it has ever told anyone is in a log you can audit." Skeptics don't trust perfection — they've been burned by claims of it. They trust accountability.

SAY IT LIKE THIS

The pitch to a skeptical CCO is not "trust it." It's: here is its own bad news, here are the receipts, try to break it, watch the inspector work — and here's the one time it was wrong and how we caught it. I'm not asking for trust. I'm handing over the means of verification.

17 · Question 4 — What does it prove, and what does it not?

What it proves

- **Trustworthy plain-English answers over sales data are buildable in take-home scope.** Direct answers, real evidence, honest flags — running code, not slides.
- **Hallucinated numbers can be made structurally impossible**, not just unlikely: keep the model out of the numeric path, and veto its wording against computed results.
- **Honesty can be a feature, not a caveat.** Refusals, red badges, and "insufficient data" verdicts are product moments — the demo's strongest ones.
- **The approach finds real problems.** Five data findings worth real money — including \$373,000 counted in two quarters at once — surfaced by the tool's own checks, not by luck.
- **The safety doesn't belong to a vendor.** Two different AI providers, one-file swap, zero change to any guarantee.

What it does not prove

- **Scale.** 180 rows of clean-ish CSV, not millions of live CRM records. The design ports; the performance story is unproven.
- **Open-ended language understanding.** Eight question types, honestly labeled. It is not "ask anything" — by design.
- **Live-system messiness.** Real Salesforce data fails in more ways than two snapshots can show. What carries over is the posture: detect, disclose, downgrade.
- **Prediction.** It measures and compares; it does not forecast — and it says so rather than inventing probabilities.
- **That it can't be wrong.** It proves something better: that being wrong is detectable, attributable, and cheap to fix. The F4 story is the evidence — found by human audit after all tests passed, automated ever since.

SAY IT LIKE THIS

One sentence for each side: it proves a leader can get a direct, evidenced, honestly-flagged answer from their own data in seconds — and it does not prove scale, open-ended language, or prediction, all of which are next steps, not features I'd fake today.

18 · Likely questions, short answers

Each answer here is two to four sentences — the spoken length. Longer versions live in the Presenter's Guide, chapter 12.

"How do I know it isn't making numbers up right now?"

Numbers never come from the AI — they come from plain code, tested against values I checked by hand. The AI's sentences pass an inspector that rejects any number the code didn't produce, and the check is printed on screen. Best of all: pick any number you see, and I'll show you its rows and its arithmetic, live.

"Why only eight questions? Why can't I ask anything?"

Because "anything" requires the AI to invent the analysis, and an invented analysis can be invisibly wrong — which is exactly what happened to you before. A short menu of audited analyses makes the worst case a visible mix-up instead of an invisible lie. The menu grows every week; the rule never changes.

"Why no RAG or vector database?"

RAG is for documents; your data is tables. On tables, fetching should be exact — a filter — not a similarity guess. The day you bring contracts or call notes, RAG becomes analysis number nine, built and audited like the rest.

"Is Priya Patel really underperforming?"

Unknowable from this data, and the tool says exactly that — 61% of her pipeline sits on corrupted records. Fix her seven records, re-ask, and then it will judge. A tool that refuses to grade people on bad data is protecting you, not failing you.

"Which is the real number — \$518,000 or \$145,000?"

Both are correctly computed from different premises. \$518,000 is what the current system says Q2 won; \$145,000 excludes the three deals your closed Q1 books already counted. Choosing between them is a finance-policy decision — the tool's job was making sure you know there's a decision to make.

"What happens when we fix the data?"

Every problem is re-detected from the files at each start. Fix the records and the flags — including the red grades — clear themselves. Nothing is hardcoded.

"What if the AI provider goes down mid-quarter?"

Nothing breaks. The app falls back to keyword matching and its own plain wording — we watched this happen live during a real provider outage and users saw no error. The AI is a convenience, never a dependency.

"How long to production?"

Foundation in two to three weeks: the calculator behind an API, read-only connection to your CRM data in a warehouse, sign-on and role-based access, and — the upgrade that matters most — daily snapshot journaling,

which makes the F4 date-moving problem impossible to hide rather than merely detectable. Then a three-leader pilot, where the success metric is behavioral: are leaders asking questions again?

"Did you use AI to build this?"

Yes, as a pair-programmer — the brief encourages it. The division of labor: AI accelerated the code; the design decisions, the metric definitions, the audit that caught F4, and the verification standards were mine — and every line ended up covered by the 42 tests. Using AI with guardrails to build a tool about AI with guardrails felt fitting.

"What would you do with four more hours?"

Three things: a "challenge this number" button that files a review ticket with the answer's full context; product-line analyses; and a one-click PDF export of any answer — evidence, formula, and flags — for meeting handouts.

19 • The one-minute closing speech

Memorize this. It's the whole engagement in ten sentences.

SAY IT LIKE THIS

Meridian's problem was never missing data — it was that leaders stopped asking questions, because the answers were slow and the one AI tool they tried made numbers up. I built a tool where a leader asks in plain English and gets three things: a direct answer, the actual rows behind it, and a flag whenever the answer leans on an assumption or a data problem. The AI in it has exactly two jobs — understand which approved question you're asking, and phrase the computed answer — and a validator vetoes any number the calculator didn't produce, so the old failure has no path to the screen. Along the way the tool's own checks found five real problems in the data, including \$373,000 of revenue counted in two quarters at once — which is why its very first answer grades itself red rather than pretending. It refuses what it can't support, logs everything it says, degrades gracefully when the AI is down, ships in a container that won't even build if a test fails, and comes with documentation the client's own team can run without me. It answers eight kinds of questions today, not eight thousand — that was the trade, and I'd make it again, because every miss is a visible refusal instead of an invisible lie. What it proves is that trust in AI answers can be engineered, not promised. What it doesn't prove — scale, open language, prediction — is a roadmap, not a gap I'd paper over. The last tool lost this client's trust by being confidently wrong. This one earns it back by being verifiably right — and honestly unsure.

Appendix A · The numbers you must know

NUMBER	WHAT IT IS
\$6,200,000	Q2 target across the ten reps
\$518,000 · 8.4%	Won so far in Q2 (day 32 of 91 · 35% elapsed)
\$145,000	Won counting only truly-new deals (F4 removed)
\$5,198,000 · 46 deals	Open pipeline (not yet revenue)
\$484,000	Gap remaining even if every open deal is won
\$165,000 / \$3,650,000 · 4.5%	Enterprise won / target — all of it re-dated; truly new = \$0
\$1,260,000 · 21.3%	Q1 at its own day 32 (Q2 runs at ~41% of that pace)
\$6,041,000 · 102%	Q1 final result — quarters here finish strong
77.9% → 1.28×	Q1 real win rate → pipeline coverage needed to be safe
Marsh · Park · Cole	The three at-risk reps (coverage below 1.0×
Patel · Torres · Williams	Unjudgeable — 54–67% of pipeline on corrupted records
F1: 13 deals · \$1.97M	Reused deal IDs — 38% of all open pipeline
F4: 3 deals · \$373K	Same revenue counted in both quarters — 72% of "Q2 booked"
42 · 5 · 22/24	Tests · reconciliations per load · routing accuracy on the golden set

Full breakdowns (every rep, segment, region, finding) are in the Presenter's Guide, Appendix B.

Appendix B · Simple glossary

WORD	PLAIN MEANING
FDE	Forward Deployed Engineer — an engineer who builds solutions at the client's side.
Quota	A salesperson's sales target for the quarter, in dollars.
Pipeline	Deals being worked on that haven't closed yet. Potential money, not real money.
Attainment	Won money divided by target. 4.5% means \$4.50 won per \$100 of target.
Coverage	Open pipeline divided by the remaining gap. Below 1.0x means the target is unreachable even winning everything.
LLM	Large Language Model — the AI that understands and produces text (Claude, Gemini, GPT).
Hallucination	An AI stating something false with confidence — here, inventing numbers.
Guardrail	A safety constraint that keeps an AI inside its intended job.
Deterministic	Same input, same output, every time — ordinary code, no AI. This is what makes checking possible.
RAG	Retrieval-Augmented Generation — fetching relevant documents for the AI to read before it answers. For document questions, not table questions.
Vector database	A search engine that finds text by similarity of meaning (e.g. ChromaDB). The storage behind RAG.
API	A doorway that lets other software talk to your software.
Docker / container	A sealed box holding the app and everything it needs, so it runs identically anywhere.
Observability	Being able to see what a running system is doing — logs, stats, health.
Fail closed	When checks fail, show nothing rather than something possibly wrong.
Audit log	A permanent record of every question and answer, for accountability.
Snapshot	A photo of the whole data table at one moment. We compared two and found five problems.
Reconciliation	Computing the same total two different ways and demanding they agree.

The Complete Walkthrough · Meridian Pipeline Intelligence · Sai Saran · Companions: live app (run.sh / docker compose up) · architecture diagram · guardrails spec · presenter's guide · one-page summary.